SLIIT | FACULTY OF COMPUTING | DEPARTMENT OF SOFTWARE ENGINEERING

SE3090 – Software Engineering Frameworks

Lecture 10

Flutter UI Handling, Device Features & Data Management



Year 3 • Semester 1 • 4 Credits • Duration: 3 Hours

Learning Outcomes: LO1, LO2, LO4 | Mobile App Development Frameworks

Where This Lecture Fits — Learning Outcomes

From a working app to a production app. Lecture 09 covered the building blocks. Today we add the layers a real mobile product needs: adaptive UI, robust navigation, managed state, validated input, device integration, storage, networking, tests and release.



LO1

Evaluate

Compare UI, state, storage and routing options and pick what fits the problem.



LO2

Apply & build

Build advanced Flutter features into one coherent application.



LO3

Integrate

Connect the app to REST APIs, handle data, test and prepare for release.



LO4

Select & justify

Justify packages, patterns and trade-offs for production readiness.

What You Already Know — A Quick Recap

From Lecture 09 we assume you are comfortable with:

Dart basics
Variables, null safety, functions, classes, async/await

Project structure
lib/, main.dart, pubspec.yaml, organising by folder

Widgets
Stateless vs Stateful, the build method, the widget tree

Layout
Row, Column, Stack, Container, Padding, ListView

Simple navigation
Navigator.push / pop between two screens

setState()
Local state updates that rebuild a StatefulWidget

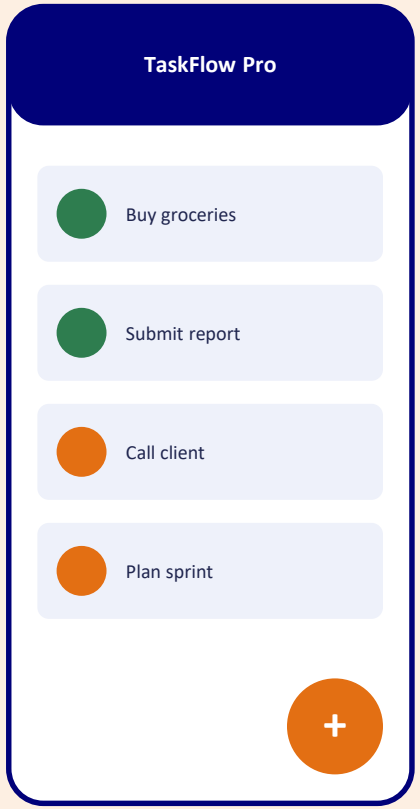
Today we do NOT re-teach these. We build directly on top of them. If any of the above feels shaky, revisit the Lecture 09 slides and the lab before the next practical — today moves quickly into advanced territory.

Why It Matters — Our Case Study: TaskFlow Pro

TaskFlow Pro

We take last week's simple task list and grow it into a realistic, production-shaped app — one feature at a time, mapped to today's topics.

- Responsive home screen (phone & tablet)
- Multi-screen navigation with tabs & a drawer
- Shared state with Provider
- A validated 'add task' form
- Attach a photo & tag a location
- Local storage + secure token storage
- Sync with a REST API + basic auth
- Unit & widget tests, then release prep



Today’s Roadmap — 3 Hours

Hour 1

Advanced UI & Navigation

Responsive/adaptive layouts • MediaQuery & LayoutBuilder • themes & UI states • accessibility & performance • navigation stack, named & declarative routing, guards • Activity + Checkpoint

Hour 2

State, Forms, Device & Data

State types & Provider/Riverpod • state-management comparison • forms & validation • plugins & permissions • camera, location, connectivity • JSON, SharedPreferences, secure storage • Activities

Hour 3

APIs, Testing & Deployment

REST integration, service layer, auth & pagination • loading/error handling • testing pyramid (unit/widget/integration) • release modes, signing & store prep • monitoring • Quiz

Two short breaks • three hands-on activities • a checkpoint and a 10-question quiz — ask questions anytime.

PART 1 OF 6

Advanced Flutter UI Handling

- Responsive & adaptive layouts for every screen size
- MediaQuery, LayoutBuilder, SafeArea and the layout toolkit
- Themes, light/dark mode and consistent styling
- Loading / empty / success / error UI states, accessibility & performance



Responsive vs Adaptive Design

Responsive

Same design fluidly resizes

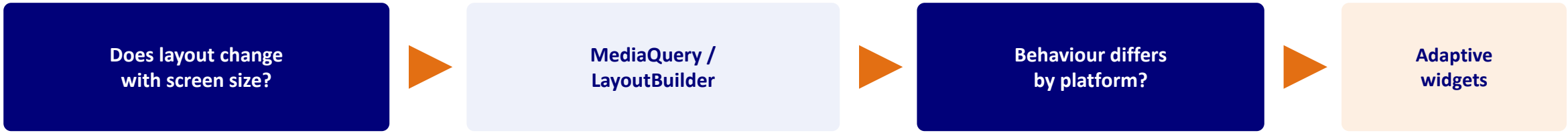
- One layout that stretches, wraps and reflows
- Driven by available width / height
- Phone → tablet → desktop from one widget tree

Adaptive

Different design per platform/device

- Distinct widgets/behaviour per platform
- e.g. Material on Android, Cupertino on iOS
- Use .adaptive constructors & platform checks

A simple decision flow



Most apps need both: responsive for size, a touch of adaptive for platform feel.

MediaQuery & LayoutBuilder

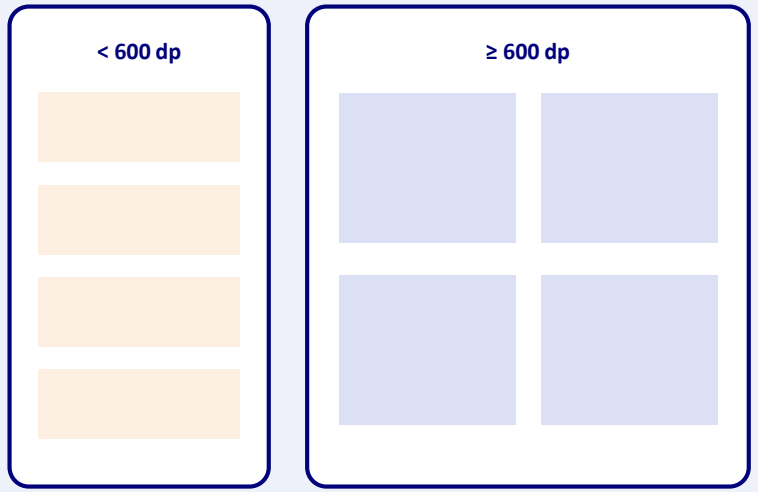
Two tools to read available space and react to it. Prefer LayoutBuilder for a widget's own box; MediaQuery for whole-screen facts.

responsive_list.dart

```dart
@override
Widget build(BuildContext context) {
  // whole-screen info: size, orientation, text scale
  final width = MediaQuery.of(context).size.width;

  return LayoutBuilder(
    builder: (context, constraints) {
      // constraints = this widget's own space
      if (constraints.maxWidth < 600) {
        return TaskListView();        // phone: 1 column
      }
      return TaskGridView(columns: 2); // tablet: 2 columns
    },
  );
}
```

What it produces



600 dp is the common phone/tablet breakpoint.

The Responsive Layout Toolkit

SafeArea

Insets content past notches, status bars and rounded corners.

Expanded / Flexible

Share remaining space inside a Row or Column proportionally.

Wrap

Flows children to the next line when they run out of width — great for chips/tags.

Stack

Layers widgets on top of each other; position with Positioned.

ListView / GridView

Scrollable lists and grids; .builder for long or infinite data.

CustomScrollView + slivers

Collapsing app bars and mixed scrolling sections in one scroll view.

Reach for these before writing manual pixel maths — they keep layouts flexible across devices.

ThemeData, Light & Dark Mode

Define colours, typography and shapes once; every widget inherits them via Theme.of(context). Ship light and dark from the same seed.

theme.dart

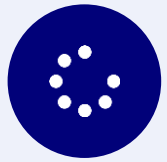
```
MaterialApp(
  themeMode: ThemeMode.system,    // follow device
  theme: ThemeData(
    colorScheme: ColorScheme.fromSeed(
      seedColor: Color(0xFF010179)),
    useMaterial3: true,
  ),
  darkTheme: ThemeData(
    colorScheme: ColorScheme.fromSeed(
      seedColor: Color(0xFF010179),
      brightness: Brightness.dark),
  ),
  home: HomeScreen(),
);
// use it:  Theme.of(context).colorScheme.primary
```

Why theme centrally

- **One source of truth for colour & type**
- Change the brand in one place
- **Free dark mode from the same seed**
- Consistent spacing & components
- **No hard-coded Colors.blue everywhere**
- Respects the user's system setting

Designing for Every UI State

Rule. Every screen that loads data has at least four states. Design all of them — a blank screen during loading or a crash on error is a real, common bug.



Loading

Spinner or skeleton while data is fetched.



Success

The data, rendered — the happy path.



Empty

No items yet — guide the user to act.



Error

Friendly message + a retry button.



FutureBuilder / state notifiers make these transitions explicit — we wire them up in the API section.

Accessibility & UI Performance

Accessibility

- Add Semantics labels for screen readers
- Respect text-scale: avoid fixed heights for text
- Tap targets ≥ 48x48 dp
- Sufficient colour contrast (don't rely on colour alone)
- Test with TalkBack / VoiceOver

Performance

- Use const constructors wherever possible
- ListView.builder / GridView.builder for long lists
- Keep setState scope small — rebuild only what changes
- Avoid deeply nested widget trees
- Use RepaintBoundary for expensive, isolated paints

Biggest beginner win: replace a Column-of-everything inside a scroll view with ListView.builder, and mark static widgets const. Both cut needless rebuilds and jank.

Activity 1 — Make This Screen Responsive

Your task (8 minutes, pairs). You are given a TaskFlow detail screen built with fixed widths (width: 350) and a non-scrolling Column. It overflows on small phones and looks lost on tablets. Redesign it to be responsive.

Apply what you've learned

- Remove fixed widths — use Expanded / Flexible
- Wrap the body in SafeArea + a scroll view
- Use LayoutBuilder to switch 1 vs 2 columns at 600 dp
- Replace literal colours with Theme.of(context)
- Make a row of tags use Wrap instead of Row

Deliverable & expected outcome

- A widget that fits a small phone with no overflow
- A sensible two-column layout on a tablet
- No hard-coded sizes or colours remaining
- One sentence: which tool fixed which problem

We'll compare two solutions on the projector. Time: ~8 minutes.

PART 2 OF 6

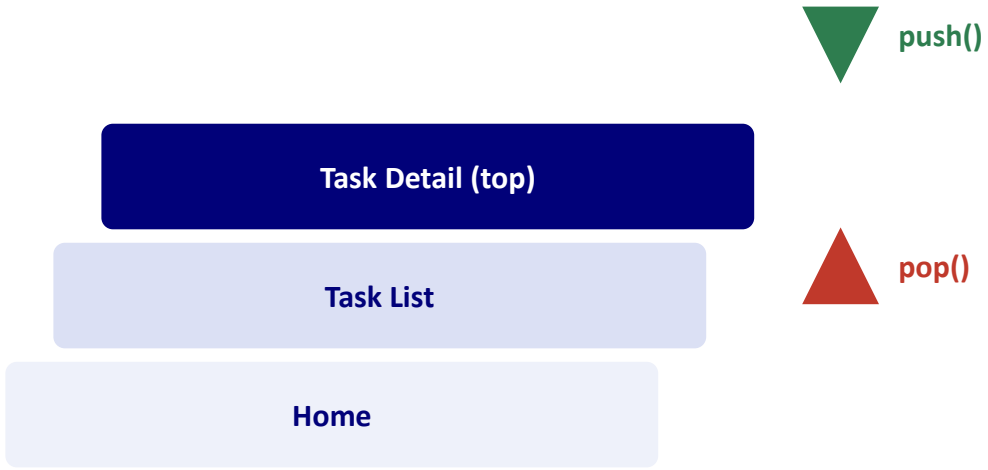
Navigation & Routing



- The navigation stack: push, pop and passing data
- Named routes vs declarative routing with GoRouter
- Bottom navigation, drawers and tabs
- Route guards, auth-aware navigation and deep linking

The Navigation Stack

Mental model. Screens are a stack of cards. push() places a new screen on top; pop() removes the top one and reveals what's beneath — exactly like the back button.



navigation.dart

```dart
// go to a new screen (push on top)
Navigator.push(
  context,
  MaterialPageRoute(
    builder: (_) => TaskDetailScreen(task)),
);

// go back (pop the top screen)
Navigator.pop(context);
```

Imperative navigation is fine for simple flows; we'll move to declarative routing for larger apps.

Named Routes & Passing Data

Register routes by name once, then navigate by string. Pass arguments in, and await a result coming back.

routes.dart

```dart
MaterialApp(
  initialRoute: '/',
  routes: {
    '/': (_) => HomeScreen(),
    '/add': (_) => AddTaskScreen(),
  },
);

// navigate by name + pass an argument
Navigator.pushNamed(
  context, '/add', arguments: 'Work');
```

return_value.dart

```dart
// read the argument on the new screen
final tag = ModalRoute.of(context)!
    .settings.arguments as String;

// return a result when popping
Navigator.pop(context, newTask);

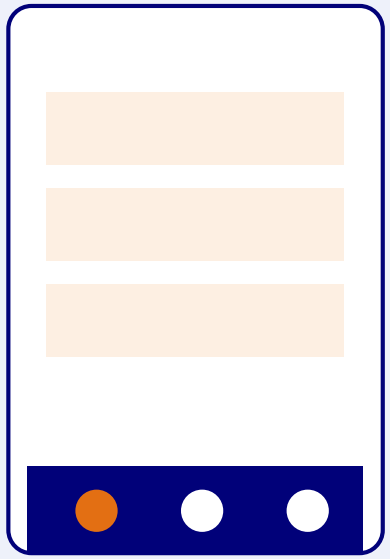
// await it back on the caller
final task = await Navigator.pushNamed(
    context, '/add') as Task?;
if (task != null) controller.add(task);
```

Two directions: pushNamed sends data forward via arguments; pop(context, value) returns a result to the screen that opened it.

Bottom Nav, Drawers & Tabs

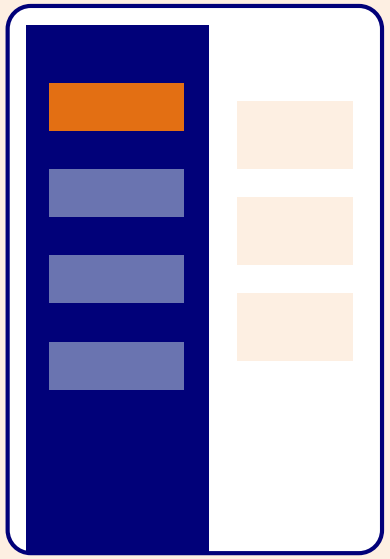
BottomNavigationBar

3–5 top-level destinations



Drawer

Many secondary destinations



TabBar

Switch peer views in a section



Combine them: a bottom bar for top-level sections, tabs inside a section, a drawer for the long tail (settings, help, logout).

Route Guards & Auth-Aware Navigation

Some screens must be protected. A guard checks a condition (logged in?) before a route renders and redirects if it fails.



After login, redirect back to the originally requested screen.

guard.dart (GoRouter redirect)

```
GoRouter(
  redirect: (context, state) {
    final loggedIn = context.read<AuthService>().isLoggedIn;
    final goingToLogin = state.matchedLocation == '/login';
    if (!loggedIn && !goingToLogin) return '/login';    // block + send to login
    if (loggedIn && goingToLogin)  return '/';          // already in → go home
    return null;                                        // no redirect needed
  },
  routes: [ /* ... */ ],
);
```

Declarative Routing with GoRouter

Declarative routing maps URLs to screens. It scales better than named routes and supports deep links, web URLs and typed parameters.

router.dart

```dart
final router = GoRouter(
  routes: [
    GoRoute(path: '/', builder: (_, __) => Home()),
    GoRoute(
      path: '/task/:id',                  // path param
      builder: (context, state) {
        final id = state.pathParameters['id']!;
        return TaskDetail(id: id);
      },
    ),
  ],
);

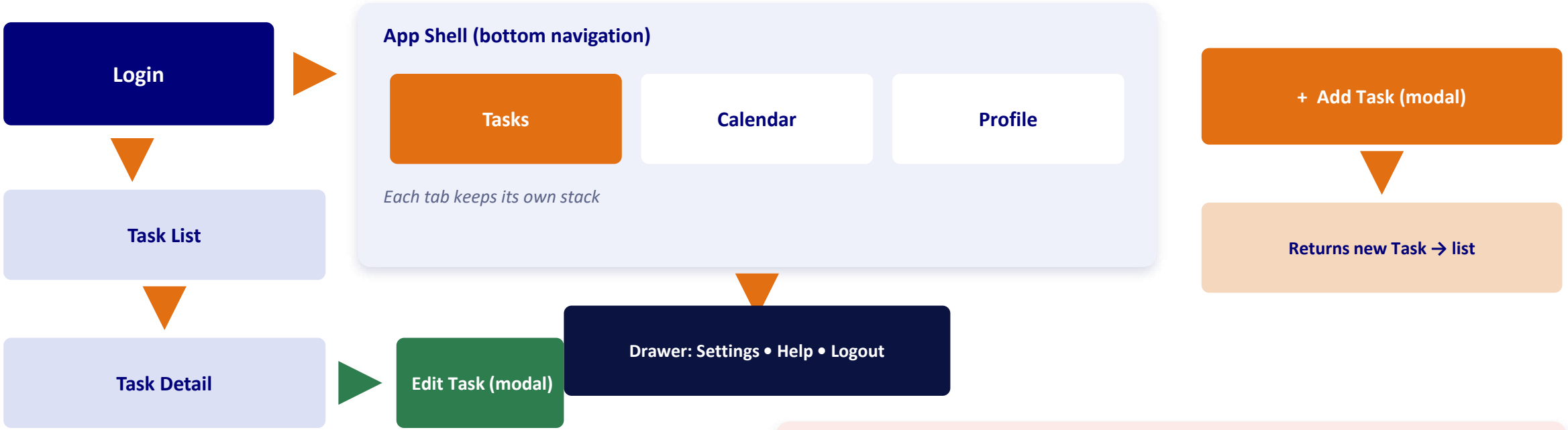
// navigate declaratively
context.go('/task/42');
```

Why GoRouter

- **URL-based & typed routes**
- Path params: /task/:id
- **Built-in redirect for guards**
- Deep links open the right screen
- **Same routes for mobile & web**
- Cleaner than deep push/pop chains

Deep linking: tapping taskflow://task/42 (or a web URL) opens Task 42 directly — the router resolves the path.

TaskFlow Pro — Navigation Flow



Pattern: bottom bar for top-level areas, modals for create/edit, a drawer for the long tail, GoRouter + a redirect guard tying it together.

Guarded: every screen except Login requires a valid token. An expired token redirects to Login, then returns the user to where they were. The + button opens a modal that returns the created task to the list.

PART 3 OF 6

State Management & Forms

- What state is — ephemeral (local) vs application (shared) state
- Why excessive setState() hurts, and how to lift state up
- Provider + ChangeNotifier in practice, with a Riverpod look
- Building robust, validated forms for task and expense entry



What Is Application State?

Definition. State is any data that can change while the app runs and that the UI depends on. When state changes, the affected UI must rebuild to reflect it.

Ephemeral (local) state

Lives in one widget • setState

- A checkbox toggle or switch value
- The currently selected tab index
- Animation progress, a text field's focus
- Safe to lose when the widget is disposed

Application (shared) state

Shared across screens • a state manager

- The logged-in user and auth token
- The task list shown on several screens
- App settings (theme, language)
- Must persist for the whole session

The deciding question: “Does any OTHER widget need this data?” No → ephemeral (setState). Yes → application state (Provider / Riverpod).

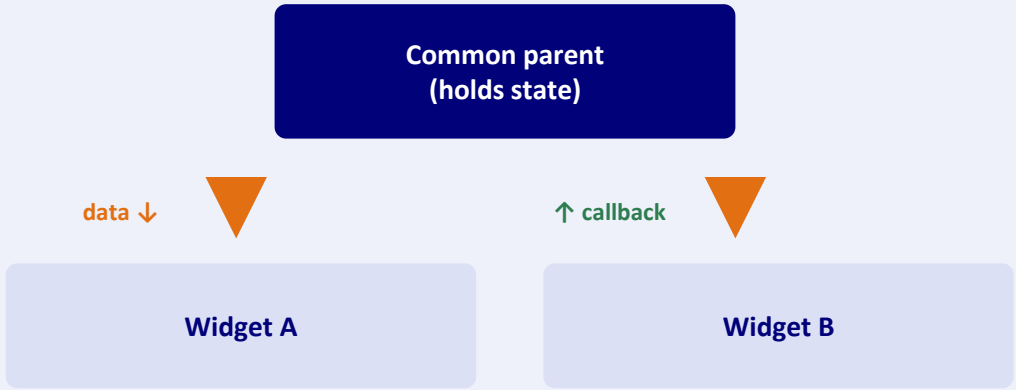
When setState() Isn't Enough

Problems with excessive setState()

- **Rebuilds the whole widget subtree, not just what changed**
- State is trapped inside one widget — siblings can't see it
- **“Prop drilling”: passing data and callbacks down many levels**
- Business logic gets tangled into the UI
- **Hard to test logic without building widgets**
- Two widgets showing the same data can fall out of sync

A first solution is **lifting state up** to a common parent.

Lifting state up



Both children now read and update the same state through the parent — they stay in sync.

Better still: lift it all the way to a state manager so any screen — not just nearby widgets — can share it.

Provider + ChangeNotifier Architecture

Provider is the recommended baseline: a **ChangeNotifier** holds the state, and widgets that depend on it rebuild only when it calls notifyListeners().

The layers

ChangeNotifierProvider
creates & exposes the model

▼

TaskModel extends ChangeNotifier
state + notifyListeners()

▼

Consumer
(task list)

Consumer
(badge count)

Listeners subscribe; each rebuilds when notified.

The update cycle

1. User taps “Add Task”

▼

2. model.add(task)

▼

3. notifyListeners()

▼

4. Consumers rebuild → UI updates

Provider in Practice

Three steps: make a ChangeNotifier model, provide it above the tree, then read it in the UI with Consumer.

task_model.dart

```dart
// 1. the model holds state + notifies
class TaskModel extends ChangeNotifier {
  final List<Task> _tasks = [];
  List<Task> get tasks => List.unmodifiable(_tasks);

  void add(Task t) {
    _tasks.add(t);
    notifyListeners();   // rebuild listeners
  }
}

// 2. provide it once, above the widgets
ChangeNotifierProvider(
  create: (_) => TaskModel(), child: const MyApp());

// 3. read it in the UI
Consumer<TaskModel>(
  builder: (context, model, _) =>
    Text('${model.tasks.length} tasks'));
```

What to notice

There are **three main steps**.

Step 1: Create a model that extends ***ChangeNotifier***.
The model stores the shared state, such as the task list.
Keep the list private using **_tasks**.

Update the state through methods such as add().

After changing the state, call **notifyListeners**().

Step 2: Use ***ChangeNotifierProvider*** to provide the model to the app.

Step 3: Use **Consumer<TaskModel>** to listen to the model and update the UI.

- **context.watch() listens for changes.**
- **context.read() accesses the model without listening.**

Riverpod — A Modern Alternative

Same mental model as Provider, with compile-time safety and easier testing. Providers are declared as top-level objects.

providers.dart

```dart
// 1. declare a provider (global, testable)
final taskProvider =
  ChangeNotifierProvider((ref) => TaskModel());

// 2. wrap the app once
ProviderScope(child: const MyApp());

// 3. read it in a ConsumerWidget
class TaskCount extends ConsumerWidget {
  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final model = ref.watch(taskProvider);
    return Text('${model.tasks.length} tasks');
  }
}
```

Why teams pick it

- **Compile-safe: no “provider not found” at runtime**
- Read state without a BuildContext
- **Trivial to override providers in tests**
- Works outside the widget tree (services, logic)
- **Scales cleanly as the app grows**

For SE3090 we use Provider as the baseline; Riverpod is the natural next step — the concepts transfer directly.

State Management at a Glance

Approach	Style	Learning curve	Boilerplate	Best for
Provider	InheritedWidget-based	Low	Low–Medium	Most apps — our baseline
Riverpod	Provider, compile-safe	Low–Medium	Low	Testable, scalable apps
BLoC / Cubit	Events & streams	Medium–High	Higher	Large, complex, event-driven
GetX	All-in-one toolkit	Low	Very low	Rapid prototyping (use with care)

There is no single best. All four solve the same problem — separate state from UI. Pick by app complexity, team familiarity and testing needs. We standardise on Provider.

Choosing & Using State Well

Decision factors

- **App size & complexity**
- How much state is truly shared
- **Team familiarity with the approach**
- Testability requirements
- **Async complexity (streams, queues)**
- Tolerance for boilerplate

Best practices

- **Keep UI and business logic separate**
- Expose read-only views; mutate only via methods
- **Rebuild the smallest widget possible (Consumer / Selector)**
- Avoid one giant “god” model — split by concern
- **Prefer immutable state — build a new object, don't mutate**

Immutable state. Instead of editing an object in place, create a copy with the change (copyWith). Changes become explicit and traceable, and rebuilds stay predictable.

Activity 2 — Pick the Right State Approach

Your task (6 minutes, individually). For each scenario decide setState, Provider, or a heavier solution — and justify your choice in one short line.

Scenarios

1. A single checkbox's on/off inside one widget
2. The logged-in user, needed on five screens
3. A three-tab screen remembering its selected tab
4. A cart shared app-wide that recomputes totals
5. A large app with complex async flows and a big team

Expected answers

- **1 → setState (ephemeral)**
- **2 → Provider (shared)**
- **3 → setState (local UI)**
- **4 → Provider / Riverpod**
- **5 → BLoC or Riverpod**

Full reasoning in the speaker notes. Time: ~6 minutes + 3 to discuss.

Form Architecture: Form, Keys & Controllers

A Form groups fields under a GlobalKey<FormState>. TextFormField carries its own validator; controllers read and set the text.

task_form.dart

```dart
final _formKey = GlobalKey<FormState>();
final _title = TextEditingController();

Form(
  key: _formKey,
  child: Column(children: [
    TextFormField(
      controller: _title,
      decoration: const InputDecoration(
        labelText: 'Task title'),
      textInputAction: TextInputAction.next,
      validator: (v) =>
        (v == null || v.isEmpty) ? 'Required' : null,
    ),
    ElevatedButton(
      onPressed: () {
        if (_formKey.currentState!.validate()) {
          // all validators passed → save the task
        }
      },
      child: const Text('Save')),
  ]));
```

The four pieces

- **Form + GlobalKey<FormState>**

  Handle to validate(), save() and reset() every field at once

- **TextFormField**

  Is used for user input and supports a built-in validator.

- **TextEditingController**

  lets us read, set, or clear the text entered by the user.

- **textInputAction / FocusNode**

  help move the user smoothly from one field to the next.

When the user clicks **Save**, call validate().

If all validators pass, then we can continue and save the task.

Validation: Required, Email & Cross-Field

Keep validators as small, reusable, pure functions — out of the widget. Each returns null when valid, or an error message string.

validators.dart

```dart
// each returns null = valid, else an error
String? required(String? v) =>
  (v == null || v.trim().isEmpty) ? 'Required' : null;

String? email(String? v) =>
  (v != null && v.contains('@')) ? null : 'Invalid email';

String? minLen(String? v, int n) =>
  (v != null && v.length >= n) ? null : 'Min $n chars';

// cross-field: confirm must equal password
String? confirm(String? v, String pwd) =>
  v == pwd ? null : 'Passwords do not match';

// use it in the field
TextFormField(validator: (v) => email(v));
```

Common rules

- Required / not-empty
- Email & phone format
- Password strength & length
- Numeric range, date in range
- Cross-field (confirm = password)

Client-side ≠ secure. Validation here is for fast UX. Always re-validate on the server — a client can be bypassed.

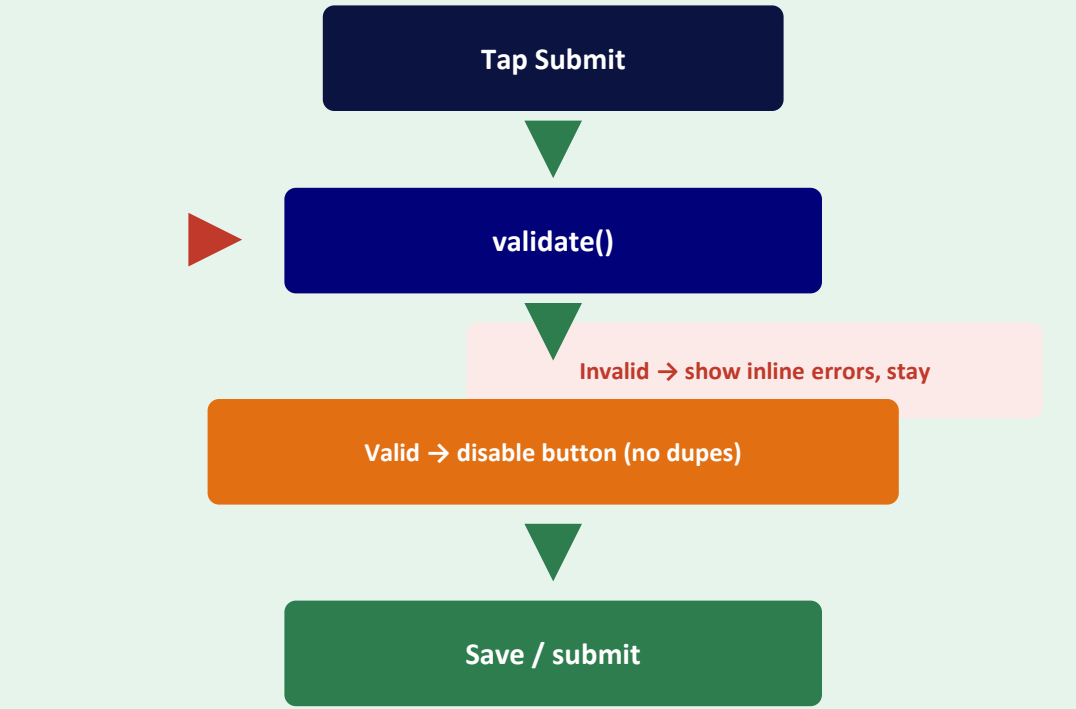
Inputs, Pickers & the Submission Flow

Beyond text fields

- Dropdown
- Switch
- Checkbox
- Radio
- Date picker
- Time picker

Dynamic fields: add or remove rows at runtime (e.g. multiple expense items).

Submission workflow



PART 4 OF 6

Device Features & Data

- The plugin ecosystem and how to evaluate a package
- Permissions: the lifecycle, Android vs iOS, and handling denial
- Camera, image picker, location and connectivity in practice
- Data models, JSON, SharedPreferences and secure storage



The Flutter Plugin Ecosystem

pub.dev hosts thousands of packages. Plugins bridge Dart to native Android/iOS code so you can use real device hardware.

What plugins unlock

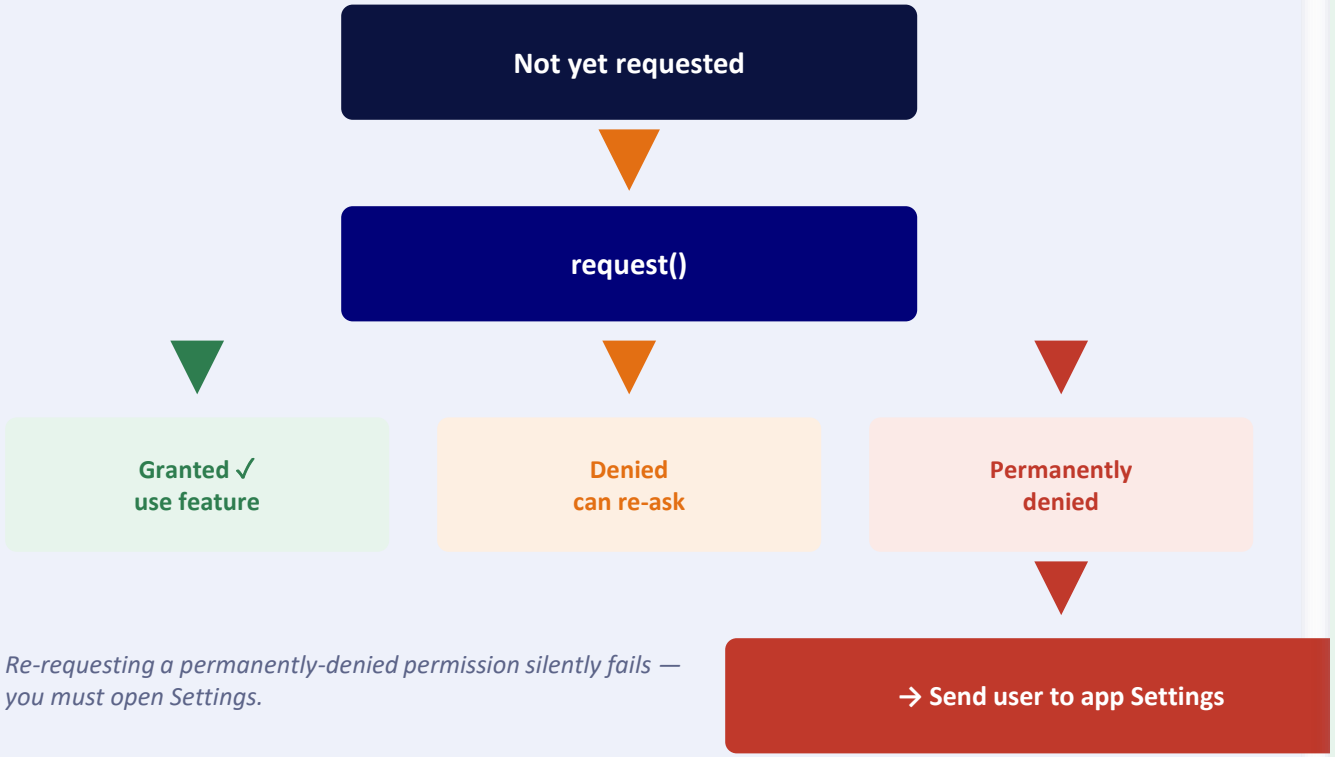
- **Camera**
- **Location / GPS**
- **Local storage**
- **Connectivity**
- **Notifications**
- **Sensors**

Before adding a package, check

- **Likes, popularity & pub points**
- Recently maintained — last publish date
- **Supports both Android and iOS (and web if needed)**
- Sound null-safety, current Flutter SDK
- **Clear docs and example code**
- Sensible permissions — not over-reaching
- **Healthy issue tracker, not abandoned**

Permission Lifecycle & Platform Differences

The permission lifecycle



Re-requesting a permanently-denied permission silently fails — you must open Settings.

Android vs iOS

- **Declare up front**
  - Android: AndroidManifest.xml
  - iOS: Info.plist + a usage-description string
- **Prompt behaviour daiffers**
  - Android can re-prompt after a denial
  - iOS prompts once; after that → Settings
- **Always handle denial gracefully**
  - Explain why, then degrade — never crash

Camera & Image Picker

The **image_picker** plugin captures a photo or picks one from the gallery. It returns an **XFile**, or **null if the user cancels.**

receipt_picker.dart

```dart
final picker = ImagePicker();

Future<void> attachReceipt() async {
  final XFile? photo = await picker.pickImage(
    source: ImageSource.camera,    // or .gallery
    maxWidth: 1200,                // downscale to save space
  );

  if (photo == null) return;       // user cancelled
  setState(() => _receiptPath = photo.path);
}
```

Get it right

- **Request camera permission first**
- Always handle null (cancel)
- **Downscale large images (maxWidth)**
- Show a preview after capture
- **file_picker covers documents too**



Preview the captured receipt before saving it to the task.

Location, GPS & Connectivity

Check the service is on and permission is granted before reading location. Watch connectivity to drive offline behaviour.

location.dart

```dart
Future<Position> getLocation() async {
  final on = await Geolocator
      .isLocationServiceEnabled();
  if (!on) throw Exception('GPS off');

  var perm = await Geolocator.checkPermission();
  if (perm == LocationPermission.denied) {
    perm = await Geolocator.requestPermission();
  }
  return Geolocator.getCurrentPosition();
}
```

connectivity.dart

```dart
// one-off check
final r = await Connectivity()
    .checkConnectivity();
final online = r != ConnectivityResult.none;

// react to changes
Connectivity().onConnectivityChanged
  .listen((r) {
    setState(() =>
      _online = r != ConnectivityResult.none);
  });
```

Same pattern everywhere: maps (google_maps_flutter), sensors (sensors_plus), notifications (flutter_local_notifications) — check permission, then call, then handle failure.

Privacy, Responsibility & Error Handling



Responsible data collection

- **Request only what the feature needs**
- Explain why before you prompt (rationale)
- **Prefer coarse over precise data where possible**
- Don't keep data longer than necessary
- **Let users decline and still use the app**



Device feature error handling

- **Permission denied → explain + offer Settings**
- Hardware missing (no camera) → fallback path
- **Service off (GPS) → prompt to enable**
- Wrap every plugin call in try / catch
- **Never crash on a denied permission**

Privacy is a requirement, not a nicety. Stores demand a privacy policy and justified permissions; regulations expect data minimisation and user consent.

Data Models & JSON Serialization

APIs and storage speak **JSON**; your app speaks **dart objects**. fromJson / toJson convert between a Map and a typed Dart model.

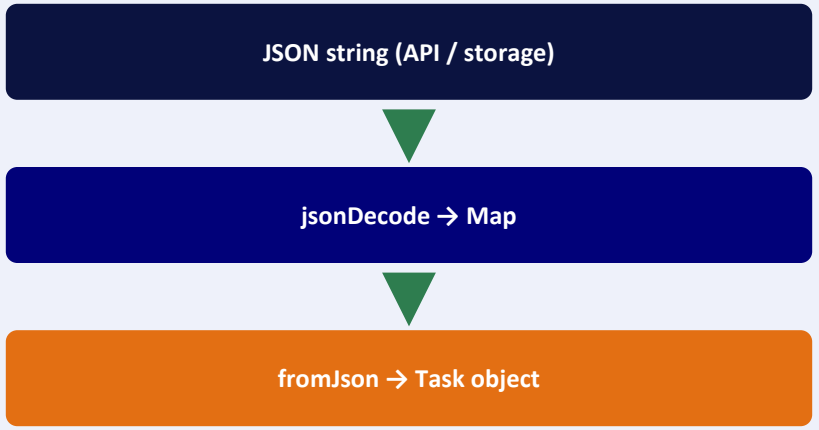
task.dart

```dart
class Task {
  final String id;
  final String title;
  final bool done;
  Task({required this.id, required this.title,
        this.done = false});

  factory Task.fromJson(Map<String, dynamic> j) =>
    Task(
      id: j['id'] as String,
      title: j['title'] as String,
      done: j['done'] as bool? ?? false,
    );

  Map<String, dynamic> toJson() =>
    {'id': id, 'title': title, 'done': done};
}
```

JSON ↔ object flow



toJson + jsonEncode reverse it on the way out.

At scale: generate fromJson/toJson with json_serializable + build_runner instead of hand-writing them — fewer typos on big models.

Local Storage: Shared Preferences & Secure Storage

SharedPreferences stores simple settings in plain text. **Secure storage** encrypts sensitive values in the OS keystore.

prefs.dart

```dart
final prefs = await
    SharedPreferences.getInstance();

// save a simple setting
await prefs.setBool('darkMode', true);

// read it back, with a default
final dark = prefs.getBool('darkMode')
    ?? false;
```

secure_storage.dart

```dart
const storage = FlutterSecureStorage();

// store an auth token (encrypted)
await storage.write(key: 'jwt',
    value: token);

// read it / clear on logout
final jwt = await storage.read(key: 'jwt');
await storage.delete(key: 'jwt');
```

Security rule. Never store tokens, passwords or keys in ***SharedPreferences*** — it's plain text. Secrets go in **secure storage** (Keychain on iOS, Keystore on Android).

Choosing the Right Storage

Option	Data kind	Encrypted	Best for
SharedPreferences	Simple key-value	No	Settings, flags, small prefs
Secure storage	Key-value secrets	Yes (OS keystore)	Tokens, passwords, keys
SQLite (sqflite)	Relational tables	No by default	Complex, related local data + queries
Hive / Isar	NoSQL objects	Optional	Fast local objects, offline cache
Remote DB (API)	Server-side	In transit (HTTPS)	Shared, multi-device source of truth

Offline & sync: local-first apps cache remote data on the device so the UI works offline, then synchronise when connectivity returns. Rule of thumb — secrets → secure storage • structured local → SQLite / Hive • settings → prefs • shared truth → API.

PART 5 OF 6

REST API Integration

- How REST and the HTTP request/response cycle work
- An API service layer: base URLs, model mapping, GET and POST
- Loading, empty and error states with FutureBuilder
- Auth headers, JWT tokens, timeouts and resilient requests



REST APIs & the HTTP Workflow

REST in one line. The app sends an HTTP request to a URL (endpoint); the server returns a response — usually JSON. Each request is independent (stateless).

HTTP methods ≈ CRUD

GET	Read data
POST	Create new
PUT	Replace
PATCH	Update part
DELETE	Remove

Status codes

2xx Success — 200 OK • 201 Created

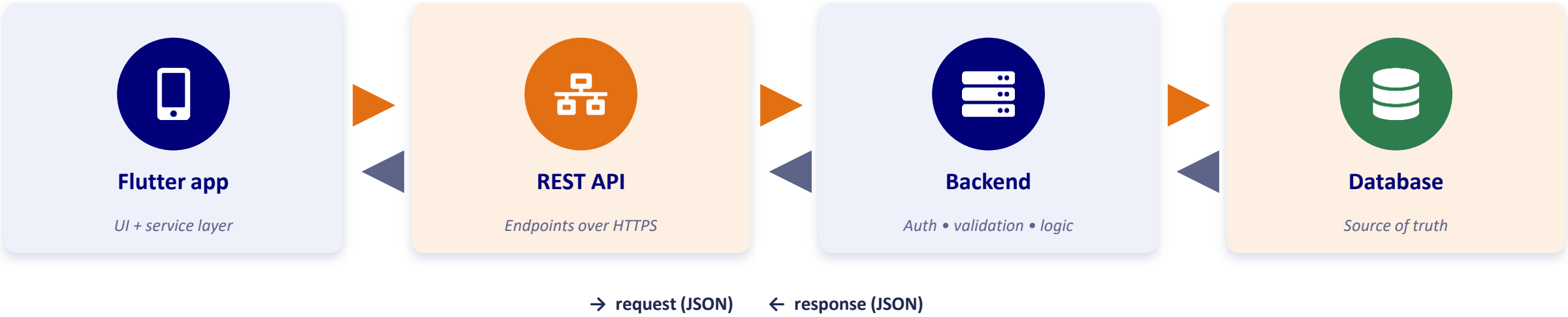
4xx Client error — 400 • 401 • 403 • 404

5xx Server error — 500 • 503

Always branch on the status code — a 200 is not guaranteed.

Flutter ↔ API ↔ Backend ↔ Database

Your app talks only to the API. The backend handles auth, validation and business logic, then reads or writes the database.



Why the app never touches the database directly: the backend enforces authentication, validates input, applies business rules and keeps credentials server-side. Putting DB access in the app would leak secrets and bypass every safeguard. The mobile app trusts nothing and verifies everything through the API.

An API Service Layer

Wrap all networking in **one service class**. The UI calls methods; the **service owns the base URL, Request headers, JSON parsing** and **error handling**.

api_service.dart

```dart
class ApiService {
  // base URL switches per environment
  static const _base = String.fromEnvironment(
    'API_URL',
    defaultValue: 'https://api.taskflow.dev');

  final http.Client _client;
  ApiService(this._client);   // inject for testing

  Uri _url(String path) => Uri.parse('$_base$path');
}

// run with:  flutter run
//   --dart-define=API_URL=https://staging.api...
```

Why a service layer

- Keeping networking in **one place** makes the code ***cleaner and easier to maintain.***
- **String.fromEnvironment('API_URL')** allows the app to use different URLs for development, staging, and production.
- This means we can switch environments without changing the source code.
- The **http.Client** is passed into the service through the constructor.
- This is **dependency injection**, and it makes the service easier to test with a mock client.
- Widgets should not build URLs or parse JSON themselves.

GET: Fetch, Map & Render

Check the status, decode the JSON list, map to typed objects — then let FutureBuilder drive the loading / empty / error / success UI.

fetch_tasks.dart

```dart
Future<List<Task>> fetchTasks() async {
  final res = await _client.get(_url('/tasks'));
  if (res.statusCode != 200) {
    throw Exception('Failed: ${res.statusCode}');
  }
  final list = jsonDecode(res.body) as List;
  return list
    .map((j) => Task.fromJson(j))
    .toList();
}
```

task_list_view.dart

```dart
FutureBuilder<List<Task>>(
  future: api.fetchTasks(),
  builder: (context, snap) {
    if (snap.connectionState !=
        ConnectionState.done) {
      return const CircularProgressIndicator();
    }
    if (snap.hasError) return ErrorView(snap.error);
    final tasks = snap.data ?? [];
    if (tasks.isEmpty) return const EmptyView();
    return TaskListWidget(tasks);
  },
)
```

This is the four-state UI from Part 1, now wired to real data: loading → error → empty → success.

Create, Update & Delete

POST a JSON body to create; **PUT/PATCH** to change; **DELETE** to remove. Check the status, and guard against duplicate submissions.

mutations.dart

```dart
Future<Task> addTask(Task t) async {
  final res = await _client.post(
    _url('/tasks'),
    headers: {'Content-Type': 'application/json'},
    body: jsonEncode(t.toJson()),
  );
  if (res.statusCode != 201) {
    throw Exception('Create failed');
  }
  return Task.fromJson(jsonDecode(res.body));
}

// update part / replace / remove
_client.patch(_url('/tasks/${t.id}'), body: body);
_client.delete(_url('/tasks/${t.id}'));
```

Don't create duplicates

- **Disable the submit button while the request is in flight**
- Track an isSubmitting / loading flag
- **A double-tap must not POST twice**
- Re-enable on success or error
- **PUT replaces the whole resource; PATCH updates part**
- Always verify the status before trusting the result

Authentication, Tokens & Failures

Read the token from secure storage, send it as a Bearer header, and handle 401, timeouts and lost connections explicitly.

authed_request.dart

```dart
Future<http.Response> authedGet(String path) async {
  final token = await storage.read(key: 'jwt');
  try {
    final res = await _client.get(
      _url(path),
      headers: {'Authorization': 'Bearer $token'},
    ).timeout(const Duration(seconds: 10));

    if (res.statusCode == 401) throw AuthException();
    return res;
  } on TimeoutException {
    throw Exception('Server timed out');
  } on SocketException {
    throw Exception('No internet connection');
  }
}
```

The JWT flow

- **Login → server returns a JWT**
- Store it in secure storage (not prefs!)
- **Attach it as a Bearer header on each call**
- 401 → token expired → refresh or re-login
- **Set a timeout on every request**
- Catch SocketException / TimeoutException
- **Never log tokens**

Robust API Integration

Scaling the data

- **Pagination — page & limit, or cursors**
- Don't fetch 10,000 tasks at once
- **Search & filter via query parameters**
- Debounce search input before calling
- **Infinite scroll loads the next page**

Resilience

- **Timeout on every call**
- Retry transient errors with backoff — not 4xx
- **Offline → serve cache, queue writes**
- Prevent duplicate in-flight requests
- **Centralise error handling in the service**

Golden rule. Keep API logic OUT of widgets. Widgets call the service; the service owns URLs, headers, parsing and errors — making it testable and reusable.

Activity 3 — Find the Bugs in This API Service

Your task (7 minutes, pairs). This getTasks() method has several real-world bugs. List every problem you can find and how you'd fix it.

buggy_service.dart

```dart
Future<List<Task>> getTasks() async {
  final res = await client.get(url('/tasks'));

  // straight to parsing...
  final data = jsonDecode(res.body);
  return data
    .map((j) => Task.fromJson(j))
    .toList();
}
```

Look for…

- Is the status code checked?
- Network / timeout handling?
- Is the auth token attached?
- Are the JSON types safe?
- What about an empty or error body?
- Where should this logic live?

Model answer in the speaker notes. Time: ~7 minutes + 4 to review together.

PART 6 OF 6

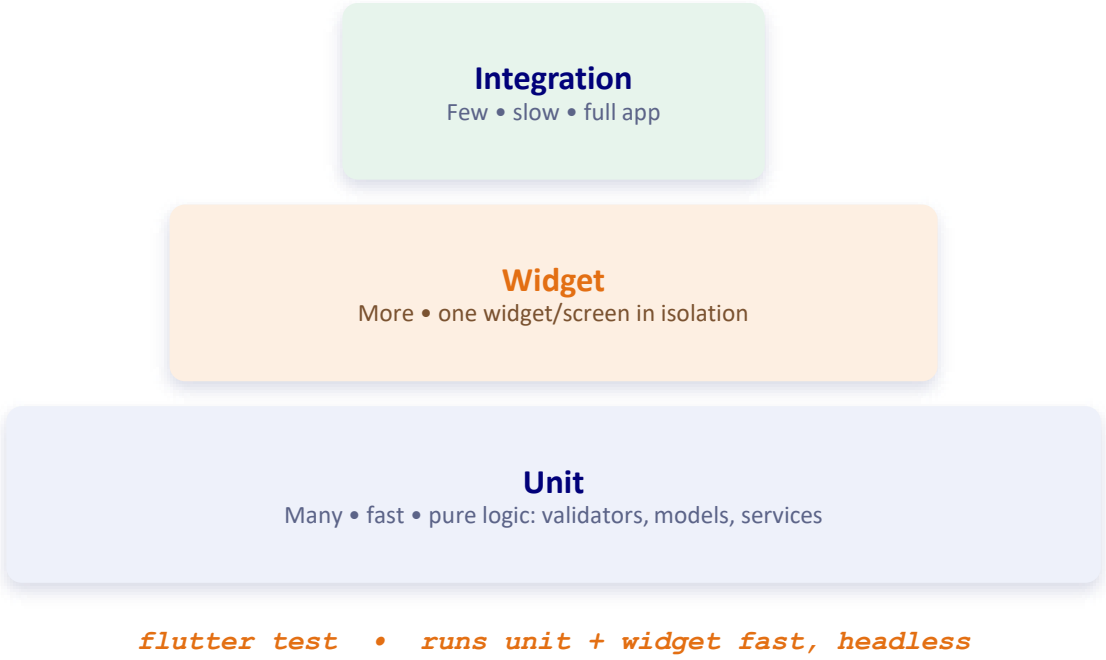
Testing & Deployment

- The Flutter testing pyramid: unit, widget and integration tests
- Writing readable unit and widget tests; mocking the API
- Debug / profile / release modes, versioning and signing
- Store submission, secrets, crash reporting and post-release monitoring



The Flutter Testing Pyramid

More tests at the bottom (fast, cheap, isolated); fewer at the top (slow, broad, high-confidence). Balance — don't invert it.



Why test a mobile app?

- Many devices, OS versions and screen sizes — you can't tap through them all by hand
- Catch regressions before users do; ship with confidence
- Tests document expected behaviour for the whole team

Testable architecture = dependency injection

Hard to test: a widget that news up its own ApiService() — the test hits the real network.

Testable: inject the service (constructor / Provider). Tests pass a fake; production passes the real one.

```
TaskListView(api: injectedService) → swap real ↔ fake
```

Unit & Widget Tests in Practice

Unit test = pure logic, no UI. **Widget test** = pump a widget, find elements, interact, assert. Same patterns test validation, navigation and storage.

validator_test.dart

```dart
void main() {
  group('email validator', () {
    test('rejects a bad address', () {
      expect(validateEmail('nope'),
             isNotNull); // returns error text
    });
    test('accepts a valid address', () {
      expect(validateEmail('a@b.com'),
             isNull);    // null = valid
    });
  });
}
```

add_task_test.dart

```dart
testWidgets('shows error on empty title',
    (tester) async {
  await tester.pumpWidget(
    const MaterialApp(home: AddTaskScreen()));

  // tap Save without typing a title
  await tester.tap(find.text('Save'));
  await tester.pump();

  expect(find.text('Title is required'),
         findsOneWidget);
});
```

Notice: the widget test verifies the SAME validator from Part 3 — unit tests prove the rule, widget tests prove it's wired to the UI.

Mocking the API, Integration & Coverage

Tests must not hit the real network. Inject a fake service to return canned data — instantly, offline, and on demand for error paths.

fake_api_test.dart

```dart
class FakeApi implements ApiService {
  @override
  Future<List<Task>> fetchTasks() async =>
    [Task(id: 1, title: 'Demo')];
}

testWidgets('renders tasks from API',
    (tester) async {
  await tester.pumpWidget(MaterialApp(
    home: TaskListView(api: FakeApi())));
  await tester.pumpAndSettle();
  expect(find.text('Demo'), findsOneWidget);
});
```

Integration tests

- integration_test package drives the REAL app on a device/emulator
- Verify full journeys: log in → add task → see it in the list
- Slower and fewer — cover only critical end-to-end flows

Coverage & where to run

- `flutter test --coverage`
- Coverage shows untested lines — a guide, not a goal; 100% proves nothing
- Emulators = fast & free for most tests; physical devices for camera, GPS, sensors & performance

Build Modes, Versioning, Signing & Secrets

Ship the RELEASE build — compiled, optimised, no debug tooling. Then version it, sign it, and keep secrets out of the binary.

Debug

Hot reload, asserts, DevTools. Slow & large. For development only.

Profile

Release-like performance + profiling on. For measuring frames/jank.

Release

AOT-compiled, optimised, stripped. The ONLY build you ship.

Versioning

version: 1.4.2+27

- 1.4.2 = version NAME shown to users
- +27 = build NUMBER, must increase every upload
- Set once in pubspec.yaml; stores reject a re-used build number

Android build

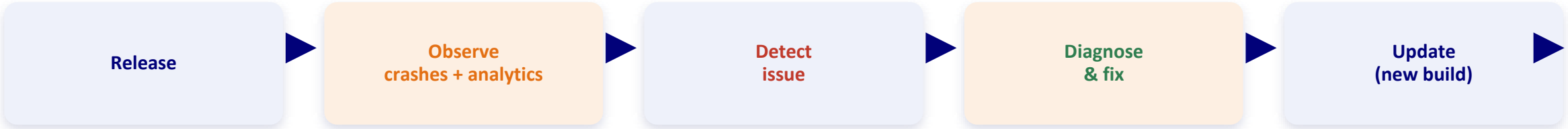
- flutter build appbundle → .aab for Google Play (optimised per device)
- flutter build apk → .apk for direct install / sideload
- Sign with a keystore (release key). Lose it = can't update the app — back it up

iOS & secrets

- iOS: archive in Xcode; needs an Apple Developer cert + provisioning profile
- **Secrets via --dart-define, NOT in source**
- `flutter build apk --dart-define=API_URL=…`
- Never commit keys/tokens; use env config per build

Submission, Monitoring & the Release Lifecycle

Shipping isn't the finish line — it's the start of a loop: release, observe, learn, improve.



↺ continuous loop — each release informs the next

Before you submit

- App icon + splash screen set; store listing & screenshots ready
- **Declare every permission and explain WHY (camera, location…)**
- Publish a privacy policy — required if you collect any user data
- Run release-build tests on real devices; check performance
- Both stores review submissions — allow days, not minutes

Observe in production

- Crash reporting (Crashlytics / Sentry) — stack traces from real users you'll never reproduce locally
- Analytics: track key flows (sign-ups, task completion) to guide decisions
- **Respect privacy: anonymise, make analytics opt-in, collect the minimum**
- Watch ratings, ANRs and crash-free rate; patch and re-release

Quick Quiz — Questions 1–5

Try each one before revealing — answers and explanations are in the speaker notes.

1 RESPONSIVE UI

Which widget reports the screen's width, height and orientation?

a) Container
b) MediaQuery
c) Scaffold

2 LAYOUT

Inside a Row, which widget makes a child expand to fill leftover space?

a) Padding
b) Center
c) Expanded

3 ROUTING

Which package gives declarative, URL-based routing with deep-link support?

a) provider
b) go_router
c) http

4 STATE MANAGEMENT

You extend ChangeNotifier and call ____ to tell listeners state changed.

(one method name)

5 STATE MANAGEMENT

"Ephemeral" state is best described as…

a) app-wide data
b) state local to one widget
c) data on a server

Quick Quiz — Questions 6–10

Covering forms, device features, storage, REST APIs, testing and deployment.

6 FORMS

A TextFormField validator returns ____ when the value is valid.

a) true
b) null
c) an empty string

7 DEVICE FEATURES

On modern Android/iOS, before opening the camera you must first…

a) nothing
b) request a runtime permission
c) restart the app

8 LOCAL STORAGE

Where should a JWT authentication token be stored?

a) SharedPreferences
b) a global variable
c) secure storage

9 REST API

In a GET handler, what must you check before jsonDecode(res.body)?

a) the body length
b) the status code
c) the device clock

10 TESTING

Which one design choice makes a widget easy to unit/widget test?

a) more setState
b) dependency injection
c) bigger build()

Lecture 10 in One Slide

From a working app to a production-ready one — here is everything TaskFlow Pro gained today.

Adaptive UI

MediaQuery, LayoutBuilder, themes, 4 UI states

Navigation

Stack, named & declarative routing, guards

State

Provider + ChangeNotifier; lift state, fewer rebuilds

Forms

Validation, controllers, pickers, submission

Device + storage

Permissions, camera/GPS, secure storage, JSON

API + ship

REST service, auth, tests, release & monitor

Learning outcomes exercised

LO1	Evaluated UI, routing, state & storage options
LO2	Built advanced features into one app
LO3	Integrated REST APIs, data, tests & release
LO4	Justified packages & trade-offs for production

In this week's lab. Extend TaskFlow Pro: responsive home, Provider state, a validated task form, a securely-stored token, and a REST task list with loading/error states.

Before Next Time & References

Prepare for the lab & next session

- Have the Flutter SDK plus an emulator or a real device ready to run
- Re-open last week's TaskFlow project — you'll evolve it into TaskFlow Pro
- Skim the pub.dev pages for provider, go_router, http, shared_preferences and flutter_secure_storage
- Revisit today's three activities — responsive screen, state choice, and the buggy API service
- Bring one design decision from your own app where you're unsure which option to pick
- **Come to the lab having attempted the responsive home screen — we build from there**

References & further reading

Flutter documentation
docs.flutter.dev — UI, navigation, testing & deployment guides

Dart language
dart.dev — language tour, null safety, async

pub.dev
Package docs: provider, go_router, http, dio, image_picker, geolocator, connectivity_plus

Secure storage & prefs
flutter_secure_storage and shared_preferences package READMEs

Testing guide
docs.flutter.dev/testing — unit, widget & integration

Material 3
m3.material.io — theming & adaptive design

Flutter Apprentice
Kodeco — practical project-based Flutter book

Thank you — see you in the lab.